

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 18
“Mesin Abstrak”



DISUSUN OLEH:

REZKY FARREL

109082500203

S1 IF-13-02

DOSEN:

Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2024/2025

SOAL LATIHAN

1.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    Left    int
    Right   int
    IsDouble bool
}

type Dominoes struct {
    Cards [28]Domino
    Count int
}

func newDominoes() Dominoes {
    var deck Dominoes
    idx := 0
    for left := 0; left <= 6; left++ {
        for right := left; right <= 6; right++ {
            deck.Cards[idx] = Domino{
                Left:    left,
                Right:   right,
                IsDouble: left == right,
            }
            idx++
        }
    }
    deck.Count = idx
    return deck
}

func kocokKartu(deck *Dominoes) {
    rand.Seed(time.Now().UnixNano())
    for i := deck.Count - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        deck.Cards[i], deck.Cards[j] = deck.Cards[j], deck.Cards[i]
    }
}

func ambilKartu(deck *Dominoes) Domino {
    if deck.Count == 0 {
        return Domino{Left: -1, Right: -1, IsDouble: false}
    }
}
```

```

    deck.Count--
    return deck.Cards[deck.Count]
}

func gambarKartu(card Domino, suit int) int {
    if suit == 1 {
        return card.Left
    }
    return card.Right
}

func nilaiKartu(card Domino) int {
    return card.Left + card.Right
}

func printDomino(card Domino) string {
    return fmt.Sprintf("[%d|%d]", card.Left, card.Right)
}

func main() {
    deck := newDominoes()
    fmt.Printf("Awal set domino: %d kartu\n", deck.Count)

    kocokKartu(&deck)
    fmt.Println("Deck sudah dikocok")

    fmt.Println("Mengambil 5 kartu pertama:")
    for i := 0; i < 5; i++ {
        card := ambilKartu(&deck)
        fmt.Printf("Kartu %d: %s, nilai=%d, sisi1=%d, sisi2=%d, balak=%t\n",
            i+1, printDomino(card), nilaiKartu(card), gambarKartu(card, 1),
gambarKartu(card, 2), card.IsDouble)
    }

    fmt.Printf("Kartu tersisa dalam deck: %d\n", deck.Count)
}

```

Output:

```

PS C:\Users\asus\Documents\Laprak Farrel\Modul18> go run "c:\Users\asus\Documents\Laprak Farrel\Modul18\CodeRunnerFile.go"
Awal set domino: 28 kartu
Deck sudah dikocok
Mengambil 5 kartu pertama:
Kartu 1: [1|4], nilai=5, sisi1=1, sisi2=4, balak=false
Kartu 2: [1|5], nilai=6, sisi1=1, sisi2=5, balak=false
Kartu 3: [2|2], nilai=4, sisi1=2, sisi2=2, balak=true
Kartu 4: [3|3], nilai=6, sisi1=3, sisi2=3, balak=true
Kartu 5: [4|4], nilai=8, sisi1=4, sisi2=4, balak=true
Kartu tersisa dalam deck: 23
PS C:\Users\asus\Documents\Laprak Farrel\Modul18>

```

REZKY FAR

+

File

Edit

View

REZKY FARREL

109082500203

Ln 2, Col 13

25 character

Plain text

170%

Deskripsi Program:

Program tersebut mensimulasikan satu set kartu domino standar yang terdiri dari 28 kartu. Program mendefinisikan struktur Domino untuk menyimpan nilai sisi kiri, sisi kanan, dan informasi apakah kartu tersebut merupakan kartu balak (kedua sisinya sama). Struktur Dominoes digunakan untuk menyimpan seluruh kartu dalam sebuah deck beserta jumlah kartunya. Fungsi newDominoes membuat set domino lengkap dari kombinasi angka 0 sampai 6, sedangkan fungsi kocokKartu mengacak urutan kartu menggunakan bilangan acak.

SOAL LATIHAN

2.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    Left    int
    Right   int
    IsDouble bool
}

type Dominoes struct {
    Cards [28]Domino
    Count int
}

func newDominoes() Dominoes {
    var deck Dominoes
    idx := 0
    for left := 0; left <= 6; left++ {
        for right := left; right <= 6; right++ {
            deck.Cards[idx] = Domino{Left: left, Right: right, IsDouble: left ==
right}
            idx++
        }
    }
    deck.Count = idx
    return deck
}

func kocokKartu(deck *Dominoes) {
    rand.Seed(time.Now().UnixNano())
    for i := deck.Count - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        deck.Cards[i], deck.Cards[j] = deck.Cards[j], deck.Cards[i]
    }
}

func ambilKartu(deck *Dominoes) Domino {
    if deck.Count == 0 {
        return Domino{Left: -1, Right: -1}
    }
    deck.Count--
    return deck.Cards[deck.Count]
}
```

```

func gambarKartu(card Domino, suit int) int {
    if suit == 1 {
        return card.Left
    }
    return card.Right
}

func nilaiKartu(card Domino) int {
    return card.Left + card.Right
}

func printDomino(card Domino) string {
    return fmt.Sprintf("[%d|%d]", card.Left, card.Right)
}

func samaGambar(card Domino, value int) bool {
    return card.Left == value || card.Right == value
}

func galiKartu(deck *Dominoes, target Domino) []Domino {
    var drawn []Domino
    if deck.Count == 0 {
        return drawn
    }

    targetValues := []int{target.Left, target.Right}
    for deck.Count > 0 {
        card := ambilKartu(deck)
        drawn = append(drawn, card)

        for _, val := range targetValues {
            if samaGambar(card, val) {
                return drawn
            }
        }
    }
    return drawn
}

func sepasangKartu(a, b Domino) bool {
    return nilaiKartu(a)+nilaiKartu(b) == 12
}

func main() {
    deck := newDominoes()
    kocokKartu(&deck)

    target := Domino{Left: 4, Right: 2}
    fmt.Printf("Target kartu: %s\n", printDomino(target))

    drawn := galiKartu(&deck, target)
    fmt.Printf("Mengambil %d kartu sampai cocok:\n", len(drawn))
}

```

```

    for i, card := range drawn {
        fmt.Printf("  %d. %s (nilai=%d)\n", i+1, printDomino(card),
nilaiKartu(card))
    }

    a := Domino{Left: 6, Right: 0}
    b := Domino{Left: 3, Right: 3}
    fmt.Printf("Sepasang %s dan %s -> %t\n", printDomino(a), printDomino(b),
sepasangKartu(a, b))
}

```

Output:

```

PS C:\Users\asus\Documents\Laprak Farrel\Modul18> go run "c:\Users\asus\Documents\Laprak Farrel\Modul18\soal2.go"
Target kartu: [4|2]
Mengambil 1 kartu sampai cocok:
  1. [4|6] (nilai=10)
Sepasang [6|0] dan [3|3] -> true
PS C:\Users\asus\Documents\Laprak Farrel\Modul18>

```

REZKY FARREL
109082500203

Deskripsi Program :

Program tersebut mensimulasikan permainan kartu domino dengan berbagai operasi pada satu set domino standar yang terdiri dari 28 kartu. Program membuat deck domino lengkap, kemudian mengacak urutannya menggunakan fungsi kocokKartu. Selain menyediakan fungsi untuk mengambil kartu, menampilkan kartu, dan menghitung nilai kartu, program juga memiliki fungsi samaGambar untuk memeriksa apakah suatu kartu memiliki angka tertentu pada salah satu sisinya. Fungsi galiKartu digunakan untuk mengambil kartu satu per satu dari deck hingga ditemukan kartu yang memiliki angka yang sama dengan salah satu sisi kartu target. Pada bagian utama program, kartu target ditentukan sebagai [4|2], lalu program menampilkan semua kartu yang diambil sampai ditemukan kartu yang cocok. Selain itu, fungsi sepasangKartu digunakan untuk memeriksa apakah dua kartu domino membentuk pasangan dengan jumlah nilai kedua kartu sama dengan 12. Hasil pengambilan kartu dan pengecekan pasangan domino kemudian ditampilkan ke layar.

SOAL LATIHAN

3.

Source Code:

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

type Domino struct {
    Left    int
    Right   int
    IsDouble bool
}

type Dominoes struct {
    Cards [28]Domino
    Count int
}

type Player struct {
    Name string
    Hand []Domino
}

func newDominoes() Dominoes {
    var deck Dominoes
    idx := 0
    for left := 0; left <= 6; left++ {
        for right := left; right <= 6; right++ {
            deck.Cards[idx] = Domino{Left: left, Right: right, IsDouble: left ==
right}
            idx++
        }
    }
    deck.Count = idx
    return deck
}

func kocokKartu(deck *Dominoes) {
    rand.Seed(time.Now().UnixNano())
    for i := deck.Count - 1; i > 0; i-- {
        j := rand.Intn(i + 1)
        deck.Cards[i], deck.Cards[j] = deck.Cards[j], deck.Cards[i]
    }
}

func ambilKartu(deck *Dominoes) Domino {
```



```

    if deck.Count == 0 {
        return Domino{Left: -1, Right: -1}
    }
    deck.Count--
    return deck.Cards[deck.Count]
}

func nilaiKartu(card Domino) int {
    return card.Left + card.Right
}

func printDomino(card Domino) string {
    return fmt.Sprintf("[%d|%d]", card.Left, card.Right)
}

func dealHands(deck *Dominoes, players []Player, handSize int) {
    for i := 0; i < handSize; i++ {
        for idx := range players {
            players[idx].Hand = append(players[idx].Hand, ambilKartu(deck))
        }
    }
}

func matches(card Domino, value int) bool {
    return card.Left == value || card.Right == value
}

func findPlayableCard(hand []Domino, leftValue, rightValue int) (int, bool, bool) {
    {
        for i, card := range hand {
            if matches(card, leftValue) || matches(card, rightValue) {
                return i, matches(card, leftValue), matches(card, rightValue)
            }
        }
        return -1, false, false
    }
}

func placeCardOnTable(card Domino, leftValue, rightValue int) (Domino, int, int, bool) {
    if card.Right == leftValue {
        return card, card.Left, rightValue, true
    }
    if card.Left == leftValue {
        return Domino{Left: card.Right, Right: card.Left, IsDouble: card.IsDouble}, card.Right, rightValue, true
    }
    if card.Left == rightValue {
        return card, leftValue, card.Right, true
    }
    if card.Right == rightValue {
        return Domino{Left: card.Right, Right: card.Left, IsDouble: card.IsDouble}, leftValue, card.Left, true
    }
}

```

```

    return card, leftValue, rightValue, false
}

func handString(hand []Domino) string {
    str := ""
    for i, card := range hand {
        if i > 0 {
            str += " ";
        }
        str += printDomino(card)
    }
    return str
}

func playGapple(players []Player, deck *Dominoes) {
    table := []Domino{}
    firstCard := ambilKartu(deck)
    table = append(table, firstCard)
    leftValue, rightValue := firstCard.Left, firstCard.Right

    fmt.Printf("Kartu pertama di meja: %s\n", printDomino(firstCard))

    passes := 0
    current := 0

    for passes < len(players) {
        player := &players[current]
        fmt.Printf("%s giliran. Tangan: %s\n", player.Name,
handString(player.Hand))

        idx, canLeft, canRight := findPlayableCard(player.Hand, leftValue,
rightValue)
        if idx == -1 {
            if deck.Count > 0 {
                drawn := ambilKartu(deck)
                player.Hand = append(player.Hand, drawn)
                fmt.Printf("%s tidak bisa main, mengambil %s\n", player.Name,
printDomino(drawn))
                idx, canLeft, canRight = findPlayableCard(player.Hand, leftValue,
rightValue)
            }
        }

        if idx >= 0 {
            card := player.Hand[idx]
            card, leftValue, rightValue, placed := placeCardOnTable(card,
leftValue, rightValue)
            if placed {
                fmt.Printf("%s memasang %s. Meja sekarang: %d-%d ... %d-%d\n",
player.Name, printDomino(card), leftValue, table[0].Left, table[len(table)-
1].Right, rightValue)
                table = append(table, card)
                player.Hand = append(player.Hand[:idx], player.Hand[idx+1:]...)
            }
        }
        current = (current + 1) % len(players)
        passes++
    }
}

```

```

        passes = 0

        if len(player.Hand) == 0 {
            fmt.Printf("%s menang!\n", player.Name)
            return
        }
    } else {
        passes++
        fmt.Printf("%s tidak bisa memasang kartu dari tangan.\n",
player.Name)
    }
} else {
    passes++
    fmt.Printf("%s melewati giliran.\n", player.Name)
}

    current = (current + 1) % len(players)
}

fmt.Println("Permainan berakhir karena semua pemain melewati giliran.")
for _, player := range players {
    total := 0
    for _, card := range player.Hand {
        total += nilaiKartu(card)
    }
    fmt.Printf("%s sisa nilai tangan: %d\n", player.Name, total)
}
}

func main() {
    deck := newDominoes()
    kocokKartu(&deck)

    players := []Player{
        {Name: "Pemain 1"},
        {Name: "Pemain 2"},
    }

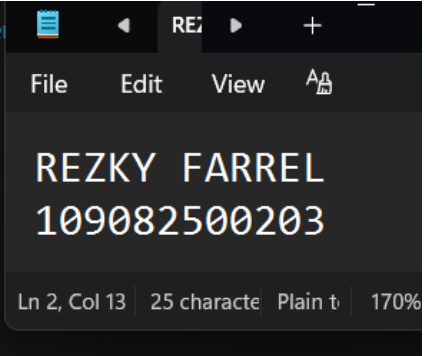
    dealHands(&deck, players, 7)
    fmt.Println("Kartu dibagikan kepada pemain.")

    playGaple(players, &deck)
}

```

Output:

```
PS C:\Users\asus\Documents\Laparak Farrel\Modul18> go run "c:\Use
3.go"
Pemain 2 tidak bisa main, mengambil [4|4]
Pemain 2 melewati giliran.
Pemain 1 giliran. Tangan: [3|6] [0|5] [0|0]
Pemain 1 tidak bisa main, mengambil [5|6]
Pemain 1 melewati giliran.
Permainan berakhir karena semua pemain melewati giliran.
Pemain 1 sisa nilai tangan: 25
Pemain 2 sisa nilai tangan: 39
PS C:\Users\asus\Documents\Laparak Farrel\Modul18> 
```



Deskripsi Program:

Program tersebut mensimulasikan permainan domino gable sederhana untuk dua pemain. Program diawali dengan membuat satu set domino standar berisi 28 kartu, kemudian mengacak urutannya dan membagikan masing-masing tujuh kartu kepada setiap pemain. Permainan dimulai dengan meletakkan satu kartu pertama di meja sebagai acuan. Pada setiap giliran, pemain akan mencari kartu di tangannya yang dapat dipasang pada salah satu ujung rangkaian domino di meja. Jika tidak memiliki kartu yang cocok, pemain akan mengambil satu kartu dari deck apabila masih tersedia. Jika tetap tidak dapat bermain, giliran dilewati. Saat pemain berhasil memasang kartu, kartu tersebut dihapus dari tangannya dan permainan berlanjut ke pemain berikutnya. Permainan berakhir ketika salah satu pemain berhasil menghabiskan seluruh kartunya dan dinyatakan sebagai pemenang, atau ketika semua pemain tidak dapat melakukan langkah lagi. Jika permainan berhenti karena tidak ada langkah yang memungkinkan, program menghitung dan menampilkan total nilai kartu yang masih tersisa di tangan setiap pemain.

SOAL LATIHAN

4.

```
package main

import "fmt"

type CharMachine struct {
    input []rune
    pos   int
}

func (m *CharMachine) start(s string) {
    m.input = append([]rune(nil), []rune(s)...)
    m.pos = 0
}

func (m *CharMachine) maju() {
    if !m.eop() {
        m.pos++
    }
}

func (m *CharMachine) eop() bool {
    return m.pos >= len(m.input) || m.input[m.pos] == '.'
}

func (m *CharMachine) cc() rune {
    if m.eop() {
        return 0
    }
    return m.input[m.pos]
}

func main() {
    m := CharMachine{}
    s := "ALFAKSALE.BETA"
    m.start(s)

    count := 0
    aCount := 0
    leCount := 0
    prev := rune(0)

    for !m.eop() {
        ch := m.cc()
        count++
        if ch == 'A' {
            aCount++
        }
        if prev == 'L' && ch == 'E' {
            leCount++
        }
    }
}
```

```

    }
    prev = ch
    m.maju()
}

fmt.Printf("Total karakter dibaca: %d\n", count)
fmt.Printf("Jumlah huruf A: %d\n", aCount)
fmt.Printf("Frekuensi LE: %d\n", leCount)
}

```

Output :

```

PS C:\Users\asus\Documents\Laprak Farrel\Modul18> go run "c:\Users\asus\Documents\Laprak Farrel\Modul18\4.go"
Total karakter dibaca: 9
Jumlah huruf A: 3
Frekuensi LE: 1
PS C:\Users\asus\Documents\Laprak Farrel\Modul18>

```

REZKY FARREL
109082500203

Ln 2, Col 13 | 25 character | Plain text | 170%

Deskripsi Program :

Program tersebut mengimplementasikan mesin karakter (character machine) untuk membaca sebuah string karakter satu per satu hingga menemukan tanda titik (.) yang berfungsi sebagai penanda akhir data. Struktur CharMachine digunakan untuk menyimpan data string dalam bentuk rune dan posisi karakter yang sedang dibaca. Pada program utama, string "ALFAKSALE.BETA" dimasukkan ke mesin karakter, kemudian setiap karakter dibaca secara berurutan sampai mencapai titik. Selama proses pembacaan, program menghitung jumlah total karakter yang dibaca, jumlah kemunculan huruf A, serta frekuensi kemunculan pasangan karakter "LE" yang berurutan. Setelah seluruh karakter sebelum titik selesai diproses, program menampilkan hasil perhitungan berupa total karakter, jumlah huruf A, dan banyaknya kemunculan pola "LE" dalam string tersebut.